

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA: ĐIỆN TỬ - VIỄN THÔNG



BÀI BÁO CÁO ĐỒ ÁN CUỐI KÌ
THỰC HÀNH TRUYỀN THÔNG DỮ LIỆU - IoT

Nhóm 6

Giảng viên hướng dẫn: Hồ Thanh Bảo

Sinh viên thực hiện: Nguyễn Vũ Nhật Thành - 22200145

Nguyễn Đức Thuận - 22200158

Lê Đăng Tâm - 22200140

Nguyễn Thế Vỹ - 22200162

Ngày 14 tháng 12 năm 2025

MỤC LỤC

MỤC LỤC	1
BẢNG PHÂN CÔNG	4
LỜI CẢM ƠN	5
TÓM TẮT ĐỒ ÁN	6
CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ PHẦN CỨNG EFR32xG21.....	7
1. Giới thiệu chung về EFR32 Series 2:	7
2. Kiến trúc Vi xử lý ARM Cortex-M33 và Bảo mật:.....	7
a. Đặc tính Kỹ thuật Cortex-M33:	7
b. Bảo mật Secure Vault và TrustZone:.....	8
3. Phân tích Subsystem Vô tuyến (Radio) 2.4 GHz:	8
4. Kit Phát triển WSTK và Radio Board BRD4180/4181:.....	9
a. Radio Board (BRD4180A/B & BRD4181):.....	9
b. Mainboard (BRD4001A/4002A) và Các Thành phần Mở rộng:	9
CHƯƠNG 2: MÔI TRƯỜNG PHÁT TRIỂN VÀ CÔNG CỤ HỖ TRỢ	11
1. Simplicity Studio 5 và Kiến trúc GSDK 4.x:.....	11
2. Công cụ Phân tích Năng lượng (Energy Profiler) và AEM:.....	12
3. Phân tích Mạng (Network Analyzer) và Giao diện PTI:	12
4. MobaXterm và Kỹ thuật Debug qua Serial:	13
5. Front-End Flutter và Android Studio IDE:	14
CHƯƠNG 3: CƠ SỞ LÝ THUYẾT CÁC GIAO THỨC - GIAO TIẾP...	15
1. UART và Giao tiếp Bất đồng bộ (VCOM)	15
2. I2C và Kỹ thuật Clock Stretching (Si7021):.....	15
3. SPI và Giao tiếp Màn hình Memory LCD	16
4. GPIO, Interrupts và Peripheral Reflex System (PRS).....	16

5. Tổng quan về cảm biến DHT11:.....	17
6. Giao thức giao tiếp One-wire (Single-bus) :.....	17
CHƯƠNG 4: KIẾN TRÚC VÀ GIAO THỨC BLUETOOTH MESH	18
1. Cấu trúc Mạng và Các loại Node	18
2. Quy trình Provisioning và Bảo mật	18
3. Mô hình Models và State.....	19
CHƯƠNG 5: TRIỂN KHAI BÀI 1.....	20
ĐỀ: Ứng dụng: Hệ thống đo nhiệt độ và độ ẩm. (Câu bắt buộc 5 Điểm)... 20	
1. Yêu cầu bài toán	21
2. Thiết kế hệ thống	22
3. Thiết kế Firmware (Code trên EFR32).....	23
4. Thiết kế phần mềm trên PC (PC App - Flutter).....	31
CHƯƠNG 6: TRIỂN KHAI BÀI 2.....	36
ĐỀ 2.....	36
1. Yêu cầu bài toán.....	37
2. Mô hình hệ thống định vị	38
3. Triển khai hệ thống.....	40
4. Kết quả thực nghiệm	46
CHƯƠNG 7: KẾT LUẬN - ĐÁNH GIÁ.....	48
1. Đối với hệ thống đo nhiệt độ - độ ẩm (Câu 1):.....	48
2. Đối với hệ thống định vị trong nhà (Câu 2):	48
3. Đánh giá hệ thống	48
4. Bảng đánh giá thành viên trong nhóm:	49

BẢNG PHÂN CÔNG

HỌ VÀ TÊN	MSSV	CÔNG VIỆC	NGÀY HOÀN THÀNH
Nguyễn Vũ Nhật Thành	22200145	Báo Cáo, Câu 1	14/12/2025
Nguyễn Đức Thuận	22200158	Câu 2, Câu 3	14/12/2025
Lê Đăng Tâm	22200140	Câu 2, Câu 3	14/12/2025
Nguyễn Thế Vỹ	22200186	PPT, Câu 1	14/12/2025

LỜI CẢM ƠN

Khi nhìn lại hành trình hoàn thành đồ án tốt nghiệp của nhóm, chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến thầy Hồ Thanh Bảo, người đã đề xuất đề tài đồ án và hỗ trợ chúng em không mệt mỏi trong suốt quá trình nghiên cứu và thực hiện.

Thầy không chỉ là người hướng dẫn chuyên môn, mà còn là nguồn cảm hứng giúp chúng em đối mặt với những thách thức và khó khăn trong quá trình nghiên cứu. Sự am hiểu sâu rộng và cách tiếp cận khoa học của thầy đã giúp chúng em mở rộng kiến thức và cải thiện kỹ năng nghiên cứu của mình.

Chúng em cũng xin cảm ơn thầy vì đã luôn sẵn lòng lắng nghe và giải đáp mọi thắc mắc, từ nhỏ nhất đến phức tạp nhất, giúp chúng em vượt qua những băn khoăn và tiếp tục tiến bộ. Những lời khuyên và hỗ trợ của thầy trong suốt quá trình làm đồ án không chỉ là nguồn động viên quan trọng mà còn là kim chỉ nam cho những quyết định và hướng đi trong nghiên cứu của nhóm.

Cuối cùng, chúng em xin gửi lời cảm ơn một lần nữa đến thầy Hồ Thanh Bảo, người thầy với lòng nhiệt huyết và tâm huyết với nghề. Những bài học và kinh nghiệm mà thầy đã chia sẻ không chỉ là đóng góp vô giá cho đồ án này mà còn là hành trang vô cùng quý báu cho con đường sự nghiệp tương lai của chúng em.

Tp. Hồ Chí Minh, Ngày....Tháng....Năm.....

TÓM TẮT ĐỒ ÁN

Báo cáo này trình bày một nghiên cứu toàn diện và chi tiết về việc ứng dụng dòng vi điều khiển (SoC) EFR32xG21 Series 2 của Silicon Labs trong các ứng dụng Internet vạn vật (IoT) hiện đại. Đồ án bao gồm phân tích sâu sắc về kiến trúc phần cứng (Cortex-M33, TrustZone, Radio 2.4GHz), hệ sinh thái phát triển (Simplicity Studio 5, GSDK 4.x), và các công cụ hỗ trợ quan trọng như MobaXterm và Network Analyzer.

Nội dung trọng tâm tập trung vào việc giải quyết hai bài toán thực tế (Bài tập 1 và Bài tập 2) thông qua việc mở rộng các giao thức giao tiếp cơ bản. Ngoài UART và I2C, báo cáo phân tích thêm về SPI (cho màn hình LCD), PWM (điều khiển độ sáng LED), và cơ chế ngắt GPIO nâng cao. Đặc biệt, phần Bluetooth Mesh được mô tả chi tiết từ lý thuyết Provisioning đến các mô hình Client-Server. Báo cáo cung cấp hướng dẫn triển khai mã nguồn, tối ưu hóa năng lượng và đo kiểm thực tế, đảm bảo tính ứng dụng cao và sự hiểu biết sâu sắc về hệ thống.

CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ PHẦN CỨNG EFR32xG21

1. Giới thiệu chung về EFR32 Series 2:

Trong kỷ nguyên IoT bùng nổ, yêu cầu về thiết bị đầu cuối không chỉ dừng lại ở kết nối mà còn đòi hỏi khả năng xử lý biên (edge processing), bảo mật phần cứng và tiết kiệm năng lượng tối đa.

Dòng sản phẩm **EFR32xG21** (Wireless Gecko Series 2) của **Silicon Labs** ra đời để đáp ứng các tiêu chuẩn khắt khe này, đặc biệt



Hình 1.1: Kit Board EFR32xG21 (Wireless Gecko Series 2) của Silicon Labs

nhắm tới các ứng dụng cấp nguồn lưới (line-powered) như bóng đèn thông minh, gateway, và trợ lý ảo, đồng thời vẫn hỗ trợ các thiết bị pin nhờ các chế độ ngủ sâu hiệu quả.

Khác với Series 1, **EFR32xG21** được tối ưu hóa mạnh mẽ về quy trình sản xuất (công nghệ 40nm) giúp tăng hiệu năng xử lý trong khi giảm dòng tiêu thụ hoạt động. SoC này hỗ trợ đa giao thức bao gồm Zigbee, Thread, Bluetooth Low Energy (BLE) 5.x và Bluetooth Mesh, tạo nền tảng linh hoạt cho các nhà phát triển.

2. Kiến trúc Vi xử lý ARM Cortex-M33 và Bảo mật:

a. Đặc tính Kỹ thuật Cortex-M33:

Hiệu năng tính toán: Với kiến trúc ARMv8-M, Cortex-M33 cung cấp khả năng xử lý vượt trội, hỗ trợ tập lệnh DSP (Digital Signal Processing) và bộ xử lý dấu phẩy động (FPU). Điều này cực kỳ quan trọng trong việc xử lý các

chồng giao thức (protocol stacks) phức tạp như Bluetooth Mesh hay thực hiện các phép toán sensor fusion ngay tại thiết bị.

Bộ nhớ: Phiên bản EFR32MG21A020F1024IM32 được trang bị 1024 kB Flash và 96 kB RAM. Dung lượng Flash lớn là yếu tố then chốt để chứa firmware OTA (Over-The-Air) bootloader cùng lúc với ứng dụng chính, đảm bảo khả năng cập nhật từ xa mà không cần bộ nhớ ngoài.

b. Bảo mật Secure Vault và TrustZone:

TrustZone: Cho phép phân chia phần cứng và phần mềm thành hai thế giới: Secure (An toàn) và Non-Secure (Không an toàn). Các tác vụ nhạy cảm như lưu trữ khóa, bootloader sẽ chạy trong vùng Secure, cách ly hoàn toàn với ứng dụng người dùng, ngăn chặn các cuộc tấn công phần mềm.

Secure Element (SE): Một hệ thống con bảo mật chuyên dụng hoạt động độc lập với lõi Cortex-M33. Nó quản lý việc sinh khóa ngẫu nhiên (TRNG), tăng tốc mã hóa AES/ECC/SHA, và đảm bảo quy trình Secure Boot, ngăn chặn việc nạp firmware giả mạo.

3. Phân tích Subsystem Vô tuyến (Radio) 2.4 GHz:

Hệ thống vô tuyến của EFR32MG21 được thiết kế để đạt hiệu suất RF cao nhất trong phân khúc:

- **Công suất phát (TX Power):** SoC tích hợp bộ khuếch đại công suất (PA) có khả năng đẩy công suất lên tới +20 dBm (100 mW). Đây là con số ấn tượng cho một con chip tích hợp, cho phép tín hiệu xuyên qua nhiều lớp tường bê tông trong các ứng dụng nhà thông minh mà không cần bộ khuếch đại ngoài (FEM).

- **Độ nhạy thu (Sensitivity):** Đạt mức -104.3 dBm tại 250 kbps (O-QPSK DSSS - Zigbee/Thread) và -97.5 dBm tại 1 Mbps (BLE). Tổng quỹ đường truyền (Link Budget) có thể lên tới hơn 124 dB, đảm bảo kết nối ổn định ngay cả trong môi trường nhiễu.
- **Dòng tiêu thụ:** Dòng đỉnh khi phát ở +10 dBm là khoảng 33.8 mA, và ở +20 dBm sẽ cao hơn đáng kể. Điều này đặt ra yêu cầu khắt khe về thiết kế nguồn (sẽ bàn kỹ ở phần Bài tập 2).

4. Kit Phát triển WSTK và Radio Board BRD4180/4181:

Để thực hiện đồ án, sinh viên sử dụng bộ Wireless Starter Kit (WSTK) bao gồm Mainboard và Radio Board. Việc hiểu rõ phân cứng này là bắt buộc để cấu hình chân (pinout) chính xác.

a. Radio Board (BRD4180A/B & BRD4181):

Radio Board là module chứa SoC EFR32MG21, ăng-ten in (Inverted-F PCB Antenna) và các linh kiện thụ động (tụ, cuộn cảm) cho mạch phối hợp trở kháng RF.

- **BRD4180A/B:** Sử dụng biến thể chip công suất cao (+20 dBm).
- **BRD4181:** Thường sử dụng biến thể công suất thấp hơn (+10 dBm) hoặc các cấu hình khác tùy revision.
- **Mapping chân:** Các chân của SoC được đưa ra hai hàng header 2.54mm ở cạnh board, khớp nối với mainboard WSTK.

b. Mainboard (BRD4001A/4002A) và Các Thành phần Mở rộng:

Mainboard đóng vai trò là mạch nạp, debug và cung cấp các ngoại vi cơ bản:

- **J-Link Debugger:** Tích hợp sẵn mạch nạp **SEGGER J-Link**, cho phép nạp code và debug qua giao tiếp **SWD** (Serial Wire Debug).
- **Si7021 (Mở rộng):** Cảm biến nhiệt độ/độ ẩm tích hợp trên mainboard, giao tiếp qua I2C.
- **Memory LCD (Mở rộng):** Màn hình **LCD 128x128** pixel của Sharp, sử dụng công nghệ Memory-in-Pixel siêu tiết kiệm năng lượng, giao tiếp qua giao thức SPI.
- **VCOM (Virtual COM Port):** Cầu nối **USB-to-UART**, cho phép máy tính nhận dữ liệu serial từ **EFR32** qua cáp **USB**.
- **Packet Trace Interface (PTI):** Một tính năng phần cứng độc quyền cho phép **Radio Board** xuất toàn bộ gói tin **RF** (raw packets) kèm metadata ra mainboard để phân tích thời gian thực.

CHƯƠNG 2: MÔI TRƯỜNG PHÁT TRIỂN VÀ CÔNG CỤ HỖ TRỢ

1. Simplicity Studio 5 và Kiến trúc GSDK 4.x:

Simplicity Studio 5 (SSv5) là IDE dựa trên Eclipse, được thiết kế tối ưu cho các dòng chip Silicon Labs. Sự chuyển dịch từ GSDK (Gecko SDK) 3.x sang 4.x mang lại cấu trúc dự án hoàn toàn mới dựa trên thành phần (component-based).



Hình 2.1: Phần mềm phát triển Simoplicity Studio 5

Project Configurator (.slcp): Trái tim của GSDK 4.x là tệp .slcp. Thay vì viết code khởi tạo thủ công, sinh viên sử dụng giao diện đồ họa để "cài đặt" các thành phần phần mềm (Software Components). Ví dụ: Muốn dùng UART, ta cài component "IO Stream: USART". Muốn dùng I2C, cài "I2C Driver".

Cơ chế autogen: Khi cài đặt một component, SSv5 sẽ tự động sinh mã nguồn vào thư mục autogen/ (ví dụ: sl_iostream_init_usart_instances.c). Sinh viên **không được sửa** các file này vì chúng sẽ bị ghi đè khi cấu hình lại.

Thư mục config: Chứa các file header (ví dụ sl_iostream_usart_vcom_config.h) cho phép định nghĩa chân (Pin), tốc độ (Baudrate), bộ đệm (Buffer). Đây là nơi sinh viên tùy biến phần cứng.

Programming Model: Hệ thống sử dụng mô hình khởi tạo sl_system_init() gọi đến các hàm khởi tạo của component, sau đó chạy

vòng lặp `sl_system_process_action()`. Code ứng dụng của người dùng được khuyến nghị đặt trong `app.c` với các hàm `app_init()` và `app_process_action()`.

2. Công cụ Phân tích Năng lượng (Energy Profiler) và AEM:

Trong các ứng dụng chạy pin (như Switch Node trong Bài 2), năng lượng là yếu tố sống còn. Energy Profiler không chỉ là một ampe kế số, nó là công cụ tương quan dòng điện với mã lệnh (Code Correlation).

AEM (Advanced Energy Monitor): Là mạch phần cứng trên WSTK, lấy mẫu dòng điện ở tốc độ 6.25 kHz (hoặc 10 kHz tùy board). Nó có dải đo rộng từ 0.1 μ A đến 95 mA.

Cơ chế hoạt động: AEM đo dòng tiêu thụ và gửi về máy tính. Đồng thời, qua chân SWO (Serial Wire Output), vi điều khiển gửi giá trị thanh ghi PC (Program Counter). Energy Profiler ghép hai luồng dữ liệu này, cho phép sinh viên biết chính xác hàm nào (ví dụ: `sl_btmesh_process_event`) đang tiêu tốn bao nhiêu năng lượng.

3. Phân tích Mạng (Network Analyzer) và Giao diện PTI:

Network Analyzer là công cụ "**sniffer**" cao cấp. Khác với các sniffer USB thông thường (chỉ bắt được gói tin trong không khí), Network Analyzer sử dụng giao diện **PTI** kết nối trực tiếp với **Radio Baseband** của **EFR32**.

Lợi ích: Bắt được cả các gói tin bị lỗi CRC, các gói tin mà ứng dụng đã lọc bỏ, và quan trọng nhất là cung cấp thời gian thực (timestamp) cực kỳ chính xác đồng bộ với hoạt động của vi điều khiển.

Ứng dụng trong đồ án: Dùng để debug quá trình Provisioning của Bluetooth Mesh, xem các gói tin Generic On/Off có thực sự được gửi đi hay không, và phân tích độ trễ của mạng.

4. MobaXterm và Kỹ thuật Debug qua Serial:

Mặc dù SSv5 có console tích hợp, MobaXterm được khuyến nghị sử dụng cho việc debug chuyên sâu.

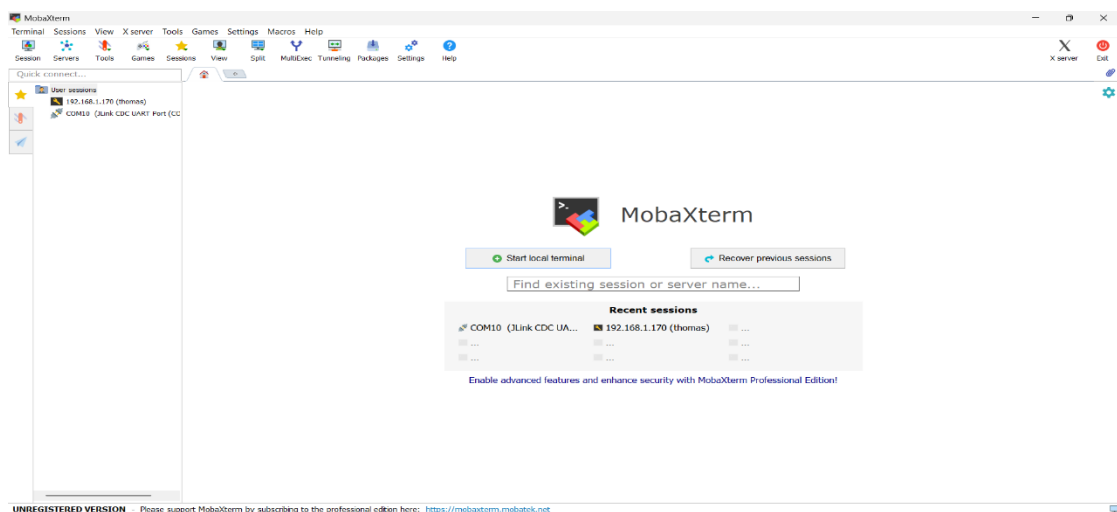
Tính năng Session: Cho phép lưu cấu hình cổng COM cho nhiều thiết bị (ví dụ: COM10 cho Light Node, COM11 cho Switch Node), giúp quản lý dễ dàng khi làm việc với nhiều node cùng lúc.



Hình 2.2: Phần mềm xem console MobaXterm

Logging: Tính năng quan trọng nhất là "Log terminal output to file". Khi chạy kiểm thử độ ổn định (stability test) qua đêm, việc ghi lại toàn bộ log ra file text giúp sinh viên truy vết lỗi (post-mortem debugging) nếu hệ thống bị treo hoặc reset bất ngờ.

Macro: Hỗ trợ gửi các chuỗi lệnh test lặp đi lặp lại một cách tự động.



Hình 2.3: Giao diện chính của MobaXterm

5. Front-End Flutter và Android Studio IDE:

Framework phát triển (Flutter): Giao diện ứng dụng (Front-End) được xây dựng dựa trên **Flutter**, bộ công cụ UI mã nguồn mở do Google phát triển. Flutter sử dụng ngôn ngữ lập trình Dart, cho phép tạo ra ứng dụng đa nền tảng (Cross-platform) chạy mượt mà trên cả Android và iOS chỉ từ một mã nguồn (codebase) duy nhất. Với kiến trúc hướng Widget linh hoạt và cơ máy dựng hình (rendering engine) riêng biệt, Flutter không chỉ giúp tối ưu hóa thời gian phát triển mà còn đảm bảo giao diện có độ phản hồi cao và hiệu năng tương đương với ứng dụng gốc (Native).



Hình 2.4: Framework phát triển Flutter

Về Môi trường phát triển (Android Studio IDE): Để hiện thực hóa mã nguồn, chúng tôi sử dụng **Android Studio** làm môi trường phát triển tích hợp (IDE) chính. Đây là công cụ cung cấp hệ sinh thái toàn diện bao gồm trình biên tập mã thông minh, hệ thống build tự động Gradle và trình quản lý thiết bị ảo (AVD Manager) để giả lập nhiều dòng máy khác nhau. Đặc biệt, Android Studio hỗ trợ đặc lực cho quy trình làm việc với Flutter thông qua các plugin chuyên dụng và tính năng 'Hot Reload', giúp lập trình viên cập nhật thay đổi giao diện tức thì và gỡ lỗi (debug) hiệu quả.



Hình 2.5: Môi trường phát triển Android Studio IDE

CHƯƠNG 3: CƠ SỞ LÝ THUYẾT CÁC GIAO THỨC - GIAO TIẾP

1. UART và Giao tiếp Bất đồng bộ (VCOM)

UART (Universal Asynchronous Receiver-Transmitter) là giao thức truyền thông nối tiếp không đồng bộ cơ bản nhất.

Khung truyền (Frame): Bao gồm 1 Start bit (mức thấp), 8 Data bits (LSB trước), tùy chọn Parity, và 1 hoặc 2 Stop bits (mức cao).

Cơ chế VCOM trên WSTK: Board Controller (BC) trên WSTK đóng vai trò chuyển đổi USB sang UART. Chân TX của EFR32 (ví dụ PA05) nối với RX của BC, và RX của EFR32 (PA06) nối với TX của BC.

Vấn đề thường gặp: Sai lệch Baudrate. Mặc định BC cấu hình VCOM ở 115200 baud. Nếu code trên EFR32 cấu hình 9600 baud, dữ liệu nhận được sẽ là rác. Cần sử dụng công cụ Simplicity Commander hoặc Admin Console để đồng bộ cấu hình này (serial vcom config speed 115200).

2. I2C và Kỹ thuật Clock Stretching (Si7021):

I2C (Inter-Integrated Circuit) sử dụng hai dây SDA (Data) và SCL (Clock) với cơ chế Open-Drain (cần điện trở kéo lên).

- **Si7021 "Hold Master Mode":** Cảm biến Si7021 có đặc điểm chuyển đổi ADC khá chậm (vài ms). Khi Master gửi lệnh đo (0xE5), cảm biến sẽ kéo chân SCL xuống mức thấp (Low) để giữ bus (Clock Stretching) cho đến khi đo xong. Vì điều khiển EFR32 phải hỗ trợ tính năng này để không bị timeout.

- **Si7021 "No Hold Master Mode":** Với lệnh 0xF5, cảm biến không kéo SCL mà sẽ trả về NACK nếu Master cố đọc dữ liệu khi chưa đo xong. Master phải liên tục thử lại (polling) cho đến khi nhận được ACK.

3. SPI và Giao tiếp Màn hình Memory LCD

Để hiển thị thông tin (Bài 1), ta sử dụng giao thức **SPI (Serial Peripheral Interface)** mở rộng. Màn hình Sharp Memory LCD trên WSTK sử dụng giao thức **3 dây (SCS, SI, SCLK)**.

Nguyên lý Memory LCD: Mỗi pixel có một mạch nhớ 1-bit cục bộ, giúp màn hình duy trì hình ảnh mà không cần làm tươi liên tục (refresh), chỉ tiêu tốn năng lượng khi thay đổi nội dung. Tuy nhiên, nó cần một tín hiệu đảo cực (VCOM Inversion - khác với VCOM của UART) tần số khoảng 1Hz để ngăn ngừa hiện tượng lưu ảnh vĩnh viễn.

Cấu hình trên EFR32: Sử dụng module USART ở chế độ Synchronous (Đồng bộ) để giả lập SPI master. Tốc độ clock có thể lên tới hàng MHz giúp vẽ hình ảnh mượt mà.

4. GPIO, Interrupts và Peripheral Reflex System (PRS)

Cấu hình GPIO: Các chân nút nhấn cần cấu hình Input Pull (hoặc Input Pull Filter để chống rung) nhằm đảm bảo mức logic xác định khi không nhấn.

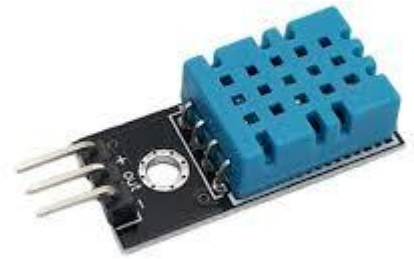
Cơ chế Ngắt EFR32 chia ngắt GPIO thành **2 vector: GPIO_ODD** và **GPIO_EVEN**. Trình điều khiển gpiointerrupt trong GSDK đóng vai trò bộ phân phối (dispatcher), xác định chân nào gây ra ngắt và gọi hàm callback tương ứng.

PRS (Mở rộng nghiên cứu): PRS cho phép các ngoại vi tương tác trực tiếp không cần CPU. Ví dụ: Nút nhấn có thể kích hoạt ADC đo thông qua PRS mà CPU vẫn đang ngủ, giúp tiết kiệm năng lượng tối đa.

5. Tổng quan về cảm biến DHT11:

Đây là dòng cảm biến tích hợp khả năng đo đồng thời nhiệt độ và độ ẩm với tín hiệu đầu ra ở dạng số (**Digital**).

Nhờ tích hợp bộ chuyển đổi analog sang digital (**ADC**) ngay bên trong, **DHT11** có khả năng chống nhiễu tốt hơn so với các loại cảm biến analog truyền thống, đảm bảo độ tin cậy cho dữ liệu thu thập được trong phạm vi hoạt động cơ bản.



Hình 3.1: Cảm biến DHT11

6. Giao thức giao tiếp One-wire (Single-bus) :

Về phương thức kết nối, **DHT11** sử dụng chuẩn giao tiếp **một dây (Single-bus)** đặc trưng.

Theo giao thức này, quá trình truyền và nhận dữ liệu giữa vi điều khiển (**Master**) và cảm biến (**Slave**) được thực hiện hoàn toàn trên một đường dẫn tín hiệu duy nhất. Việc này giúp tối ưu hóa thiết kế phần cứng và tiết kiệm tài nguyên chân GPIO cho vi điều khiển.

Tuy nhiên, để đảm bảo đường truyền hoạt động ổn định ở trạng thái nghỉ (**Idle**), thiết kế mạch bắt buộc phải sử dụng một điện trở kéo lên (**Pull-up resistor**) nối giữa chân tín hiệu và nguồn dương.

CHƯƠNG 4: KIẾN TRÚC VÀ GIAO THỨC BLUETOOTH MESH

❖ **Bluetooth Mesh** là công nghệ mạng lưới (mesh networking) hoạt động trên nền tảng **BLE**, cho phép giao tiếp nhiều-nhiều (**many-to-many**).

1. Cấu trúc Mạng và Các loại Node

Khác với WiFi (mô hình Sao - Star) hay Zigbee (Routing), Bluetooth Mesh sử dụng kỹ thuật "**Managed Flooding**" (Lũ lụt có kiểm soát).

Relay Node: Nhận gói tin và phát lại (retransmit) để mở rộng vùng phủ sóng. Sử dụng TTL (Time-To-Live) để giới hạn số lần nhảy (hop).

Proxy Node: Cho phép các thiết bị không hỗ trợ Mesh (như smartphone thường) kết nối vào mạng Mesh qua giao thức GATT.

Friend & Low Power Node (LPN): Đây là cặp quan hệ quan trọng cho Bài 2. Switch Node (chạy pin) sẽ là LPN, ngủ 99% thời gian. Light Node (chạy điện lưới) sẽ là Friend, có nhiệm vụ lưu trữ tin nhắn giúp LPN trong khi LPN đang ngủ. LPN chỉ thức dậy định kỳ (Poll) để hỏi Friend xem có tin nhắn không.

2. Quy trình Provisioning và Bảo mật

Để tham gia mạng, thiết bị phải trải qua quá trình Provisioning nghiêm ngặt :

Beaconing: Thiết bị chưa cấu hình phát gói tin Unprovisioned Device Beacon.

Invitation & Capabilities: Provisioner (Smartphone) mời và thiết bị phản hồi về khả năng của nó (số element, thuật toán hỗ trợ).

Public Key Exchange: Trao đổi khóa công khai (ECDH) để thiết lập kênh bảo mật.

Authentication: Xác thực qua OOB (Out-Of-Band) như nháy LED hoặc nhập mã số.

Distribution: Provisioner gửi NetKey (Khóa mạng), AppKey (Khóa ứng dụng) và gán địa chỉ Unicast Address.

3. Mô hình Models và State

Mọi hành vi trong Mesh được định nghĩa bởi Model.

Server Model: Chứa trạng thái (State). Ví dụ: Generic On/Off Server chứa trạng thái Bật/Tắt của đèn.

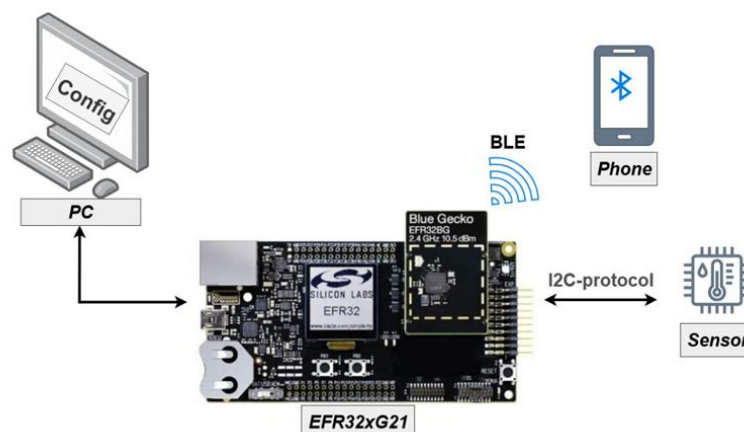
Client Model: Gửi lệnh để thay đổi trạng thái của Server. Ví dụ: Generic On/Off Client gửi lệnh SET ON/OFF từ công tắc.

Publish/Subscribe: Cơ chế gửi nhận tin nhắn. Công tắc Publish tới địa chỉ Nhóm (Group Address), Đèn Subscribe địa chỉ Nhóm đó.

CHƯƠNG 5: TRIỂN KHAI BÀI 1

ĐỀ: Ứng dụng: Hệ thống đo nhiệt độ và độ ẩm. (Câu bắt buộc 5 Điểm)

Mô tả tổng quan: Hệ thống sử dụng EFR32xG21 để đọc thông tin từ một cảm biến theo chuẩn giao tiếp I2C (cảm biến có ít nhất 2 thông tin) như: MPU6050, AHT20, MAX30102, ... hiển thị dữ liệu lên LCD, và gửi thông tin đến PC thông qua UART. Đồng thời, hệ thống sử dụng BLE để quảng bá dữ liệu đo được đến các thiết bị BLE (như smartphone). Người dùng có thể cấu hình các thông số hệ thống và xem dữ liệu đo lường thông qua giao diện trên PC viết bằng ngôn ngữ C. Sau đó lưu dữ liệu nhận được sử dụng 1 cơ sở dữ liệu tùy chọn.



Hình 5.1: Mô hình kết nối câu 1

Yêu cầu thiết bị:

- BRD4180B Radio Board (EFR32xG21 2.4 GHz 20 dBm) + Wireless Starter Kit Mainboard (BRD4001A).
- Cảm biến theo chuẩn giao tiếp I2C + dây kết nối. (Sinh viên tự chuẩn bị).
- Điện thoại thông minh hỗ trợ BLE + Ứng dụng Simplicity Connect hoặc ứng dụng hỗ trợ bluetooth tương tự.

Mô tả chi tiết:

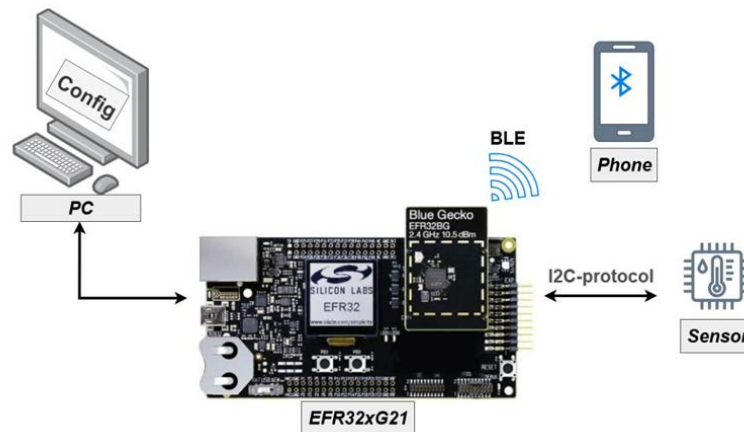
- a. PC giao tiếp với EFR32xG21 thông qua UART để thực hiện các chức năng:
 - Cấu hình: Chu kì đo thông tin cảm biến như nhiệt độ/độ ẩm, nhịp tim/nồng độ Oxy trong máu, ... chu kì quảng bá của BLE.
 - **Lấy 2 thông tin của cảm biến và lưu vào 1 cơ sở dữ liệu tùy chọn.**
 - Một số tính năng khác sinh viên tự đề xuất (không bắt buộc).
- b. EFR32xG21 đọc giá trị của cảm biến theo chu kì và dữ liệu được hiển thị trên LCD bao gồm: 2-3 thông tin của cảm biến và chu kì đọc cảm biến.
- c. EFR32xG21 sử dụng BLE để quảng ra môi trường xung quanh. Gói tin quảng bá bao gồm: Tên thiết bị, giá trị cảm biến (thông tin 1, thông tin 2).

1. Yêu cầu bài toán

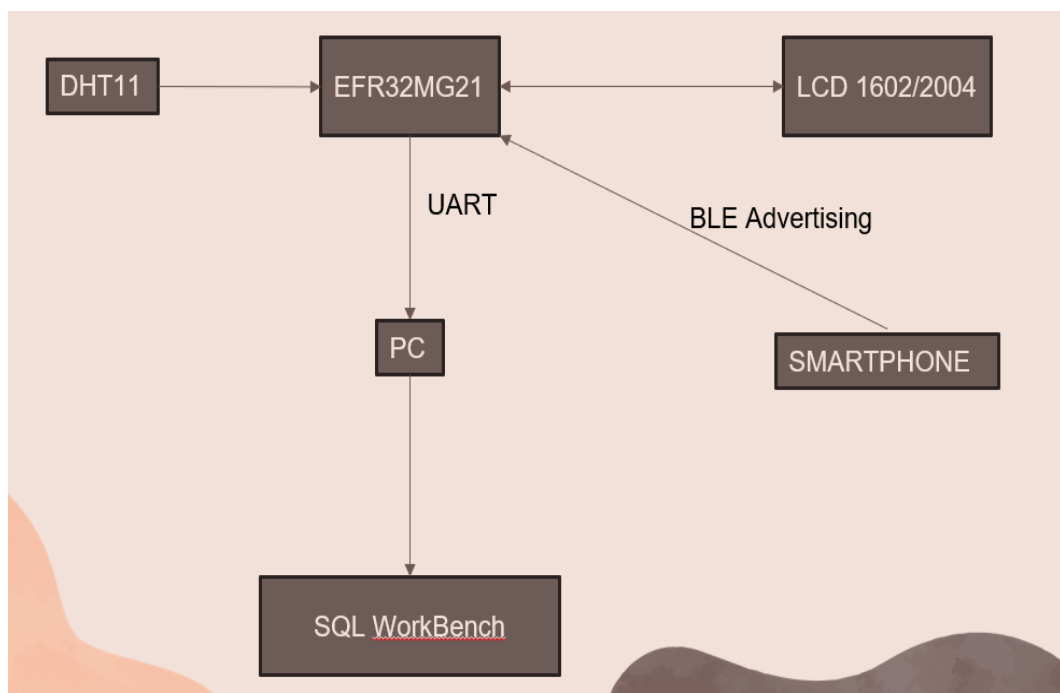
- **Mục tiêu:** Xây dựng hệ thống đo nhiệt độ, độ ẩm sử dụng EFR32xG21.
- **Chức năng:**
 - ✓ Đọc dữ liệu từ cảm biến, hiển thị thông tin lên màn hình LCD.
 - ✓ Gửi dữ liệu lên PC qua giao tiếp UART để lưu trữ và cấu hình.
 - ✓ Quảng bá dữ liệu (Advertising) qua BLE để các thiết bị di động có thể quét thấy.

2. Thiết kế hệ thống

a. Sơ đồ khối (Block Diagram):



Hình 5.2: Mô hình kết nối câu 1



Hình 5.3: Block Diagram dành cho câu 1

b. Lựa chọn phần cứng:

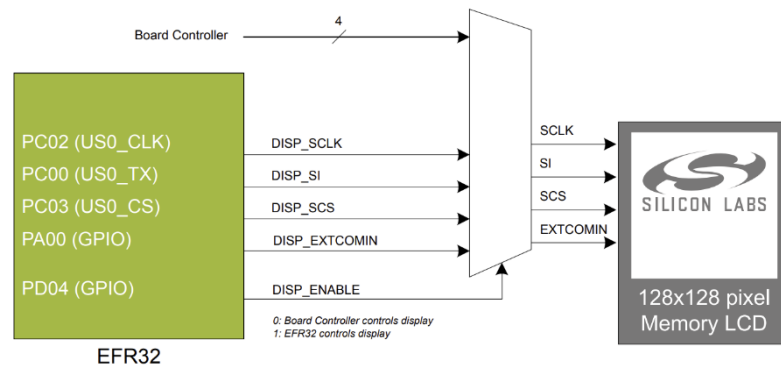
- **Kit phát triển:** EFR32xG21 (Wireless Starter Kit Mainboard BRD4001A + Radio Board BRD4180B).
- **Cảm biến:** DHT11.
- **Hiển thị:** LCD (Mem-LCD trên kit).

3. Thiết kế Firmware (Code trên EFR32)

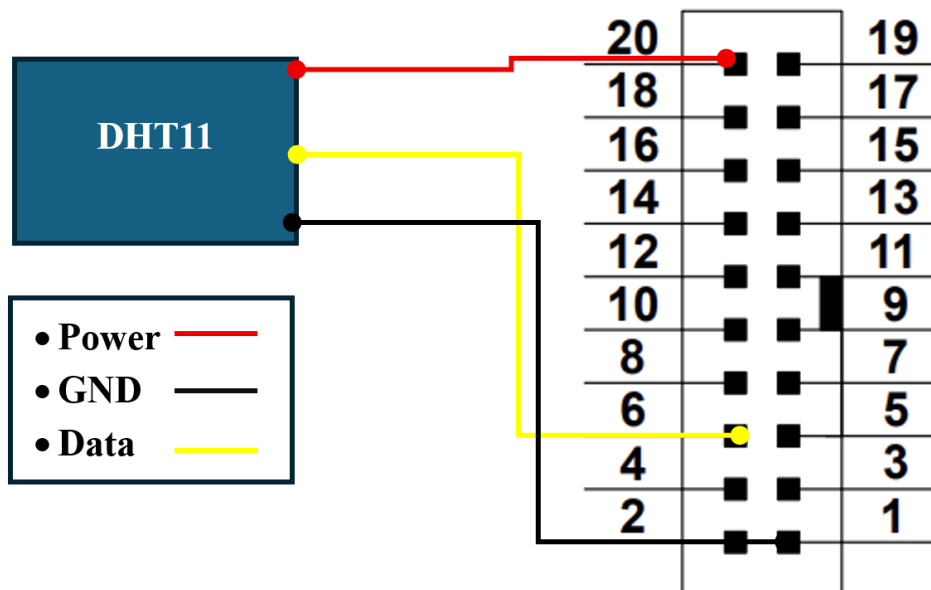
a. Cấu hình ngoại vi (Pinout & Configuration):

- Mô tả chân GPIO dùng cho DHT11.

** Tips: Sử dụng hình ảnh sơ đồ chân đã cấu hình với LCD, Ta có thể loại trừ và lấy các chân còn lại làm chân tín hiệu*

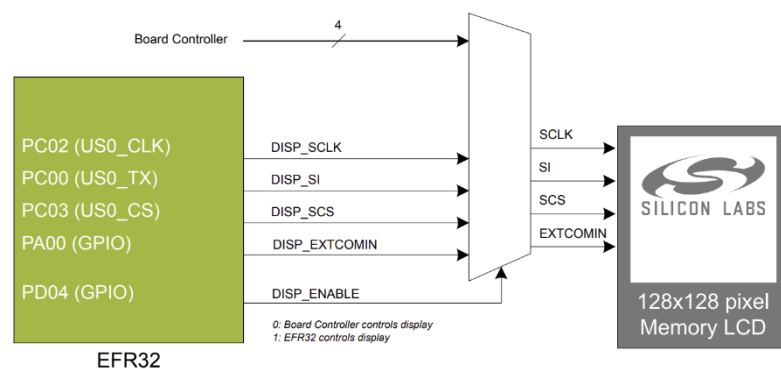


Hình 5.4: 128x128 Pixel Memory LCD



Hình 5.5: Cấu hình chân DHT11 với EXP Header

- Cấu hình UART (Baudrate, Parity...).
 - Cấu hình mặc định của VCOM khi khởi chạy ở Simplicity trên dòng kit này là: **115200 baud, 8 data bits, No parity, 1 stop bit (115200 8N1)**
- Cấu hình LCD.



Hình 5.6: 128x128 Pixel Memory LCD

b. Hoạt động chính:

- **Khởi tạo**
 - Phần này thiết lập màn hình, chờ cảm biến ổn định và khởi động bộ định thời (Timer) cũng như thiết lập Bluetooth ban đầu.

```
void app_init(void)
{
    sl_status_t sc;
    printf("\r\n--- START TRAM 1 (Nhom: 6) ---\r\n");

    init_lcd_system(); // Khởi tạo màn hình LCD (GLIB, DMD)
    sl_sleeptimer_delay_millisecond(2000); // Chờ 2s để cảm biến nóng máy

    // Khởi động Timer: Đây là trái tim của vòng lặp, nó sẽ gọi hàm update_timer_cb sau mỗi chu kỳ
```



```
    sc = app_timer_start(&update_timer, MEASURE_INTERVAL_MS,
update_timer_cb, NULL, true);
    app_assert_status(sc);
}

// --- Trong hàm sl_bt_on_event (Sự kiện Boot) ---
case sl_bt_evt_system_boot_id:
    // Lấy địa chỉ MAC
    sc = sl_bt_system_get_identity_address(&address,
&address_type);

    // Tạo Advertising Set
    sl_bt_advertiser_create_set(&advertising_set_handle);

    // Cấu hình tham số quảng bá (Interval)
    sl_bt_advertiser_set_timing(advertising_set_handle, 160,
160, 0, 0);

    // Bắt đầu quảng bá tên thiết bị
    fill_adv_packet(&sData, FLAG, COMPANY_ID, 0x00000000,
"TRAM_DHT11");
    start_adv(&sData, advertising_set_handle);
    break;
```

- **Đọc DHT11**

- ✓ Thực hiện giao thức 1-wire (Bit-banging) để lấy dữ liệu thô từ cảm biến.

```
static void update_timer_cb(app_timer_t *timer, void *data)
{
    // BẮT ĐẦU VÙNG TỐI HẠN (CRITICAL SECTION)
    // Tạm khóa ngắt để đảm bảo đo xung thời gian chính xác
    CORE_DECLARE_IRQ_STATE;
    CORE_ENTER_CRITICAL();

    DHT11_Start(); // Gửi tín hiệu Start
    uint8_t presence = DHT11_Check_Response(); // Kiểm tra phản
hồi

    if (presence) {
```

```
// Đọc lần lượt 5 byte dữ liệu
Rh_byte1 = DHT11_Read_Byte(); // Độ ẩm (nguyên)
Rh_byte2 = DHT11_Read_Byte(); // Độ ẩm (thập phân)
Temp_byte1 = DHT11_Read_Byte(); // Nhiệt độ (nguyên)
Temp_byte2 = DHT11_Read_Byte(); // Nhiệt độ (thập phân)
SUM = DHT11_Read_Byte(); // Checksum
}
```

- **Cập nhật LCD**

- Xóa màn hình cũ và vẽ các thông số mới (Tên nhóm, MAC, Nhiệt độ, Độ ẩm, Chu kỳ, Số lần đo)

```
// Xóa bộ nhớ đệm màn hình
GLIB_clear(&glibContext);

// Vẽ các dòng thông tin
GLIB_drawStringOnLine(&glibContext, "NHOM 6_DHT11", 0,
GLIB_ALIGN_CENTER, 0, 5, true);
GLIB_drawStringOnLine(&glibContext, mac_display_str, 1,
GLIB_ALIGN_CENTER, 0, 10, true);

sprintf(lcd_buf, "TEMP: %d.%d C", Temp_byte1, Temp_byte2);
GLIB_drawStringOnLine(&glibContext, lcd_buf, 3, GLIB_ALIGN_LEFT, 5,
0, true);

sprintf(lcd_buf, "HUM : %d.%d %%", Rh_byte1, Rh_byte2);
GLIB_drawStringOnLine(&glibContext, lcd_buf, 4, GLIB_ALIGN_LEFT, 5,
0, true);

sprintf(lcd_buf, "Cycle: %d ms", MEASURE_INTERVAL_MS);
GLIB_drawStringOnLine(&glibContext, lcd_buf, 6,
GLIB_ALIGN_CENTER, 0, 0, true);

sprintf(lcd_buf, "Count: %lu", measure_count);
GLIB_drawStringOnLine(&glibContext, lcd_buf, 7,
GLIB_ALIGN_CENTER, 0, 0, true);

// Đẩy dữ liệu ra màn hình thật
DMD_updateDisplay();
```

- **Gửi UART**

- Gửi dữ liệu đã xử lý lên máy tính qua cổng COM (VCOM)

```
if (presence && (SUM == (uint8_t)(Rh_byte1 + Rh_byte2 +  
Temp_byte1 + Temp_byte2))) {  
  
    // Gửi chuỗi định dạng qua UART (stdout được retarget  
vào VCOM)  
    printf("[OK] TRAM | T:%d.%d C | H:%d.%d %% | Cycle:%dms  
| Count:%lu\r\n",  
           Temp_byte1, Temp_byte2, Rh_byte1, Rh_byte2,  
MEASURE_INTERVAL_MS, measure_count);
```

- **Cập nhật gói tin BLE Advertising**

- Đóng gói dữ liệu cảm biến vào định dạng Hex (BCD) và cập nhật nội dung gói tin quảng bá mà không cần ngắt kết nối.

```
// Đóng gói dữ liệu vào biến 32-bit (4 byte)  
// Byte 3: Nhiệt độ nguyên  
// Byte 2: Nhiệt độ thập phân  
// Byte 1: Độ ẩm nguyên  
// Byte 0: ID Trạm (0x03)  
uint32_t sensor_packed = (make_visual_dec(Temp_byte1)  
<< 24) |  
                           (make_visual_dec(Temp_byte2)  
<< 16) |  
                           (make_visual_dec(Rh_byte1) <<  
8) |  
                           0x03; // ID trạm (Hoặc 0x01,  
0x08 tùy code)  
  
// Cập nhật gói tin quảng bá (User Data / Manufacturer  
Data)  
update_adv_data(&sData,      advertising_set_handle,  
sensor_packed);
```

- **Delay chu kỳ**

- Không dùng hàm delay() (blocking delay) mà dùng **App Timer** (Software Timer) để tạo chu kỳ lặp lại chính xác và tiết kiệm năng lượng.

```
#define MEASURE_INTERVAL_MS 2000 // Định nghĩa chu kỳ 2000ms
```

- **Cập nhật dữ liệu qua SQL bằng Python**

a) Khởi tạo database trên SQL Workbench

- Tạo docker với port 3306 (tùy chọn)
- Dùng port đã đăng kí với docker → Khởi tạo database trên SQL → Test connection → Done
- Viết Query tạo Tables

```
-- 1. Tạo Database nếu chưa tồn tại
CREATE DATABASE IF NOT EXISTS sensor_db;

-- 2. Chọn Database để sử dụng
USE sensor_db;

-- 3. Tạo bảng dht11_data
CREATE TABLE IF NOT EXISTS dht11_data (
    id INT AUTO_INCREMENT PRIMARY KEY,          -- Khóa chính,
    humidity FLOAT NOT NULL,                     -- Lưu độ ẩm
    temperature FLOAT NOT NULL,                 -- Lưu nhiệt độ
    cycle_count INT DEFAULT 0,                  -- Lưu số chu kỳ
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- Tự động
    lưu thời gian ghi
);

-- 4. Kiểm tra lại cấu trúc bảng (Optional)
DESCRIBE dht11_data;
```

b) Viết chương trình cho Python

▪ Khởi tạo & Cấu hình

```
#
=====
=====
# CẤU HÌNH HỆ THỐNG
#
=====
=====
SERIAL_PORT = 'COM15'    # Cấu hình cổng COM kết nối với mạch
BAUD_RATE   = 115200     # Tốc độ truyền (phải khớp với code
C)
DB_HOST      = '127.0.0.1' # Địa chỉ database
DB_USER      = 'root'
DB_PASS      = 'root'
DB_NAME      = 'sensor_db'
```

▪ Kết nối Database

```
def connect_db():
    """Hàm kết nối Database có thử lại nếu thất bại"""
    try:
        conn = mysql.connector.connect(
            host=DB_HOST,
            user=DB_USER,
            password=DB_PASS,
            database=DB_NAME
        )
        print(f"✅ [SYSTEM] Đã kết nối MySQL: {DB_NAME}")
        return conn
    # ... (Phần xử lý lỗi)
```

▪ Lưu dữ liệu

```
def save_to_db(cursor, db, humidity, temperature, count):
    """Lưu dữ liệu vào bảng """
    try:
        # Câu lệnh SQL chèn dữ liệu
```

```
        query = "INSERT INTO dht11_data (humidity,
temperature, cycle_count) VALUES (%s, %s, %s)"
        cursor.execute(query, (humidity, temperature,
count))
        db.commit() # Xác nhận lưu
        print(f"💾 [DB] Saved: T={temperature}C |
H={humidity}% | Count={count}")
        # ...
```

▪ Nhận dữ liệu UART

```
# 2. Kết nối Serial (Mở cổng)
try:
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE,
timeout=1)
    # ...

# 3. Vòng lặp chính (Đọc dữ liệu liên tục)
try:
    while True:
        if ser.in_waiting > 0:
            raw_bytes = ser.readline() # <--- ĐỌC DỮ LIỆU
TỪ MẠCH TẠI ĐÂY
            try:
                line = raw_bytes.decode('utf-8',
errors='ignore').strip()
                if line:
                    print(f"📡 [RAW]: {line}") # In dữ
liệu thô ra màn hình
```

▪ Giải mã gói tin

```
# --- Xử lý dữ liệu ---
        if "[OK]" in line:
            # Dùng Regex để "gấp" các con số
ra khỏi chuỗi văn bản
            match =
re.search(r"T.*[:\s](\d\.)+.*H.*[:\s](\d\.)+.*Count[:\s
]*(\d+)", line)
```

```
        if match:
            # Tách thành công, gán vào
            biến
            temp_val =
float(match.group(1)) # Nhiệt độ
            hum_val =
float(match.group(2)) # Độ ẩm
            count_val =
int(match.group(3)) # Số chu kỳ (Count)

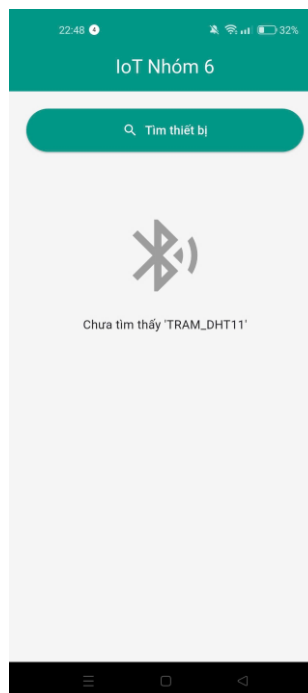
            # Gọi hàm lưu vào DB
            save_to_db(cursor, db_conn,
hum_val, temp_val, count_val)
```

▪ Quản lý chu kỳ

```
time.sleep(0.01) # Nghỉ 10ms
```

4. Thiết kế phần mềm trên PC (PC App - Flutter)

- *Ngôn ngữ/Công cụ: Dart/Flutter*
- *Giao diện người dùng (GUI):*



- **Giải mã gói tin quảng bá từ EFR32xG21:**

```
if (bytes.length < 4)
    return Text("Gói tin lỗi (Length < 4): $bytes");
// Byte 0: Nhiệt độ phần nguyên
int tInt = bcdToDecimal(bytes[0]);
// Byte 1: Nhiệt độ phần thập phân
int tDec = bcdToDecimal(bytes[1]);
// Byte 2: Độ ẩm phần nguyên
int hInt = bcdToDecimal(bytes[2]);
// Byte 3: ID Trạm
int id = bytes[3]; // ID
```

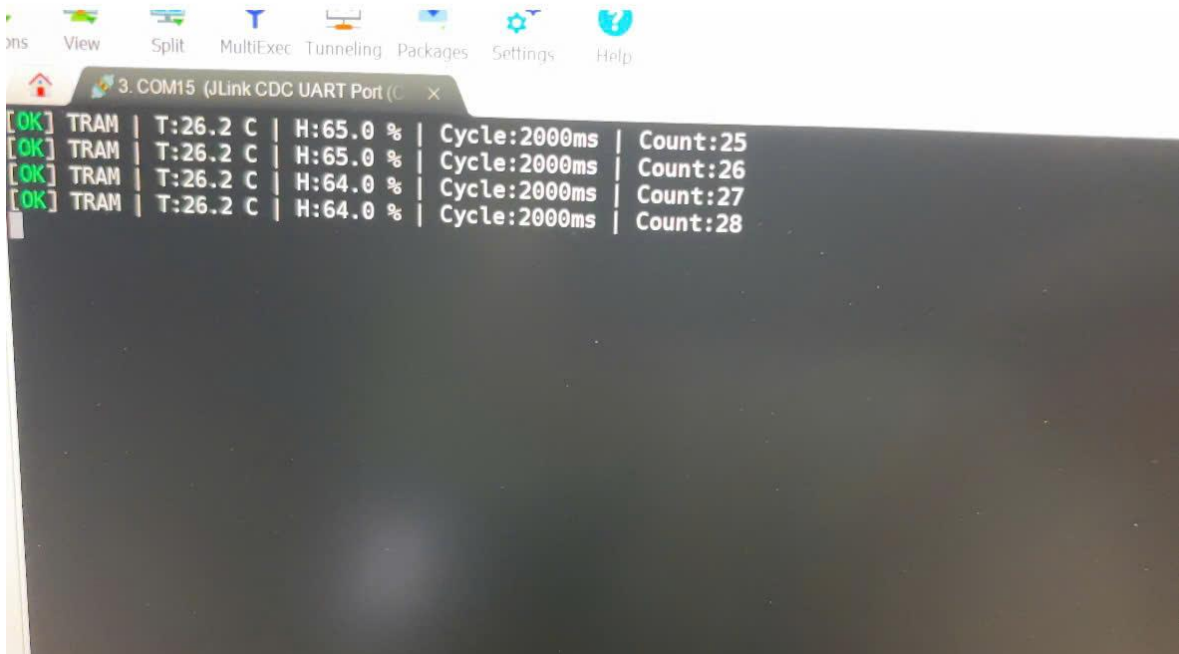
- **Chức năng:**
 - ✓ Kết nối COM Port.
 - ✓ Hiển thị đồ thị/số liệu thời gian thực.
 - ✓ Cấu hình chu kỳ đo.
 - ✓ Lưu trữ Database (mô tả dạng file csv, txt hoặc SQL).

5. Kết quả thực nghiệm

- **Kết quả trên LCD:**



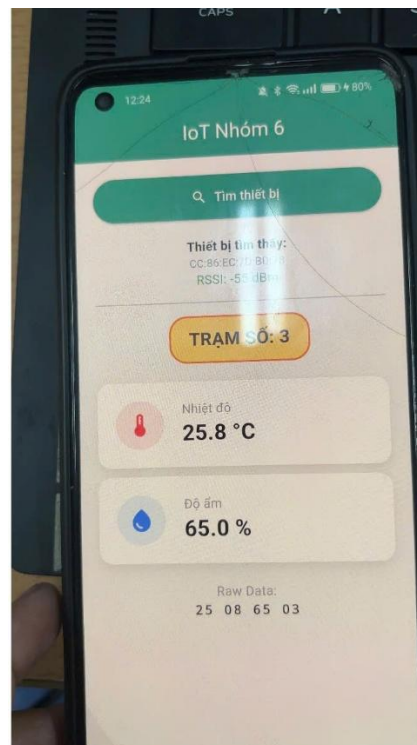
- Kết quả trên PC



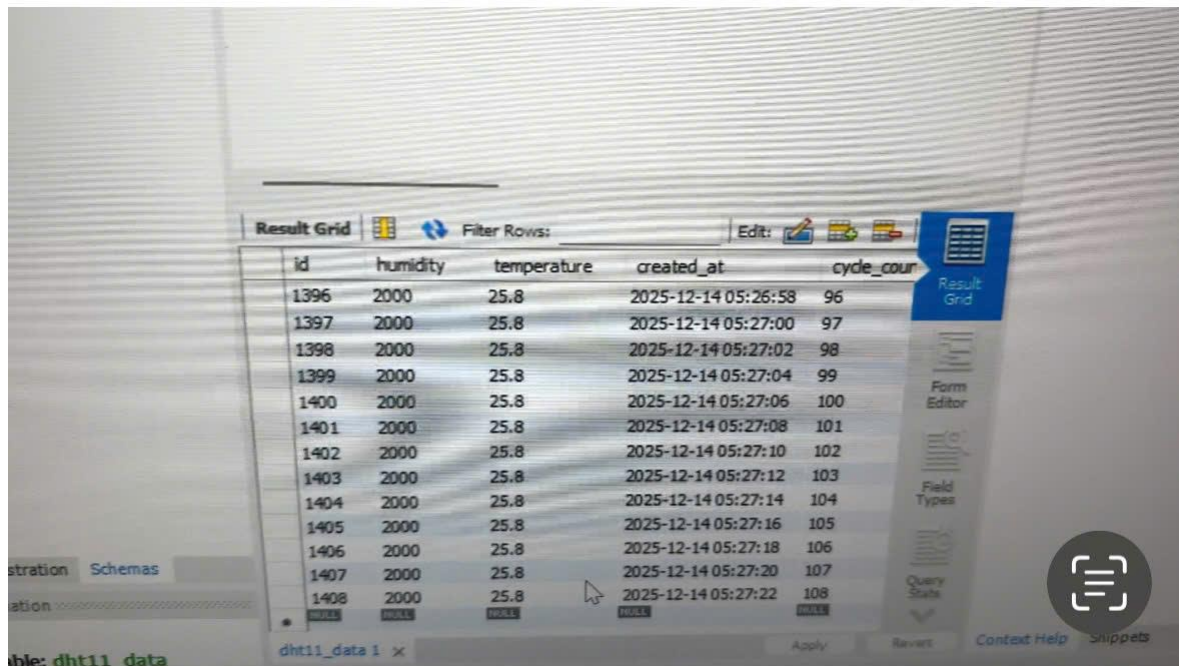
```
✓ [SYSTEM] Đã kết nối MySQL: sensor_db
✗ [SERIAL ERROR] Không thể mở cổng COM15!
  Chi tiết: could not open port 'COM15': PermissionError(13, 'Access is denied.', None, 5)

C:\Users\AD\Downloads\iot>python3 serial_to_sql.py
--- BẮT ĐẦU LOGGER (TRAM1_DHT11) TRÊN COM15 ---
✓ [SYSTEM] Đã kết nối MySQL: sensor_db
🔊 [SERIAL] Đang lắng nghe cổng COM15...
📡 [RAW]: [OK] TRAM | T:25.8 C | H:64.0 % | Cycle:2000ms | Count:96
🗄️ [DB] Saved: T=25.8C | H=2000.0% | Count=96
📡 [RAW]: [OK] TRAM | T:25.8 C | H:64.0 % | Cycle:2000ms | Count:97
🗄️ [DB] Saved: T=25.8C | H=2000.0% | Count=97
📡 [RAW]: [OK] TRAM | T:25.8 C | H:64.0 % | Cycle:2000ms | Count:98
🗄️ [DB] Saved: T=25.8C | H=2000.0% | Count=98
```

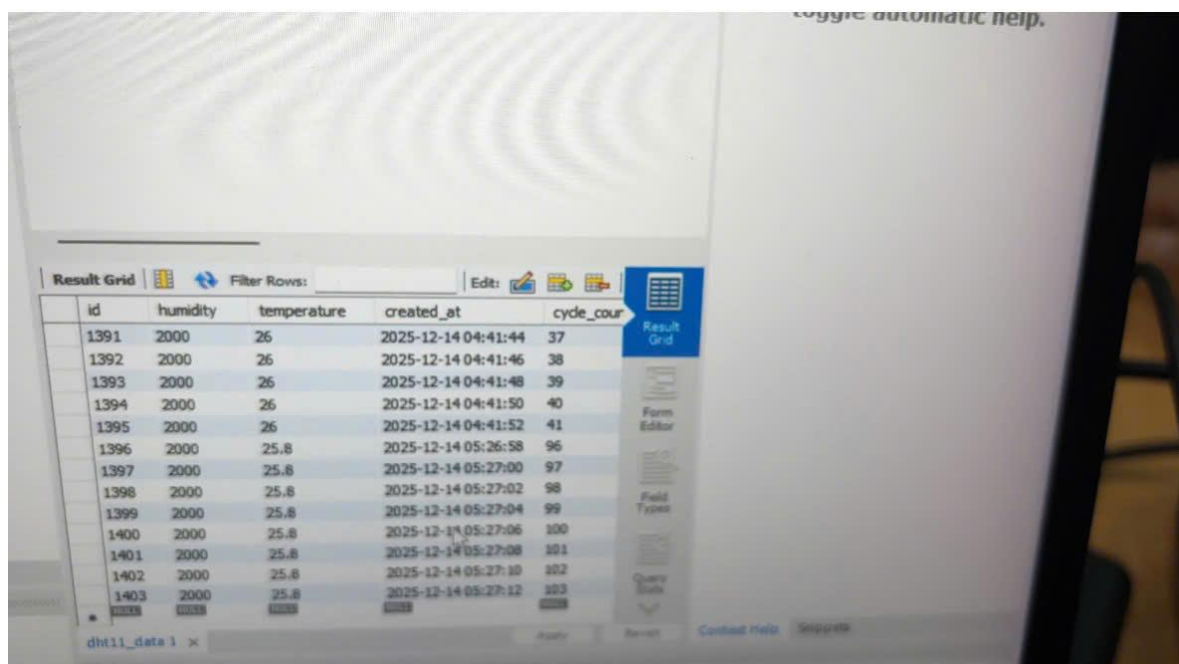
- Kết quả trên App điện thoại:



- Database:



id	humidity	temperature	created_at	cycle_count
1396	2000	25.8	2025-12-14 05:26:58	96
1397	2000	25.8	2025-12-14 05:27:00	97
1398	2000	25.8	2025-12-14 05:27:02	98
1399	2000	25.8	2025-12-14 05:27:04	99
1400	2000	25.8	2025-12-14 05:27:06	100
1401	2000	25.8	2025-12-14 05:27:08	101
1402	2000	25.8	2025-12-14 05:27:10	102
1403	2000	25.8	2025-12-14 05:27:12	103
1404	2000	25.8	2025-12-14 05:27:14	104
1405	2000	25.8	2025-12-14 05:27:16	105
1406	2000	25.8	2025-12-14 05:27:18	106
1407	2000	25.8	2025-12-14 05:27:20	107
1408	2000	25.8	2025-12-14 05:27:22	108

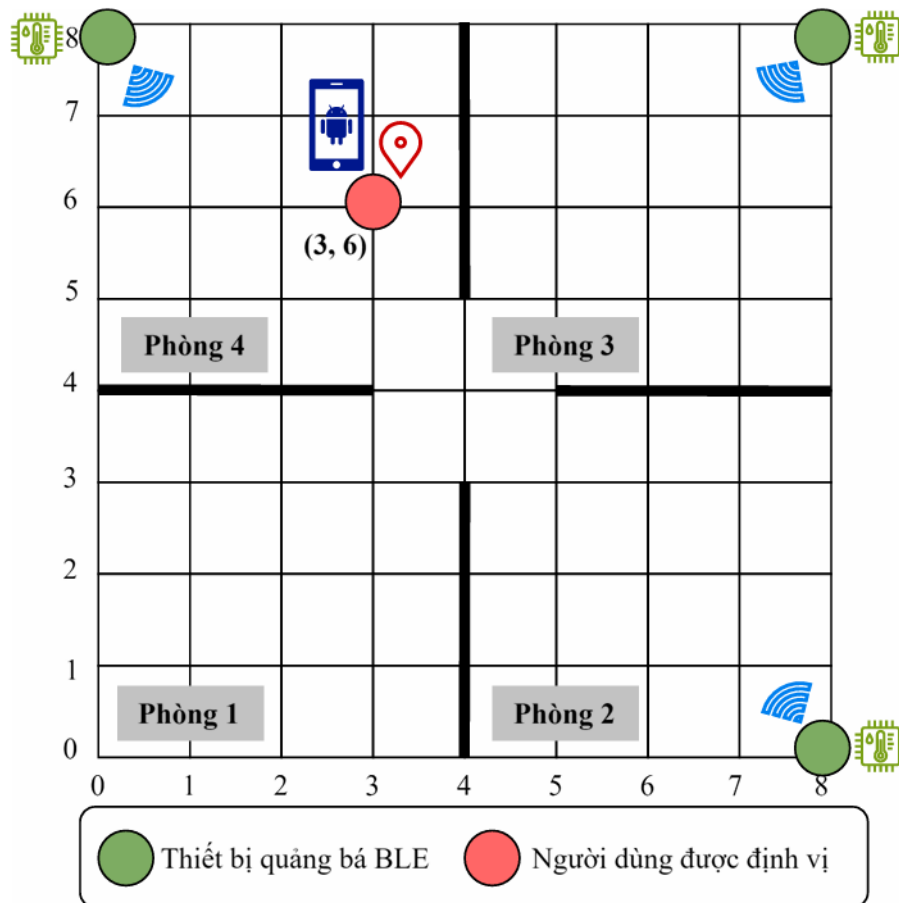


id	humidity	temperature	created_at	cycle_count
1391	2000	26	2025-12-14 04:41:44	37
1392	2000	26	2025-12-14 04:41:46	38
1393	2000	26	2025-12-14 04:41:48	39
1394	2000	26	2025-12-14 04:41:50	40
1395	2000	26	2025-12-14 04:41:52	41
1396	2000	25.8	2025-12-14 05:26:58	96
1397	2000	25.8	2025-12-14 05:27:00	97
1398	2000	25.8	2025-12-14 05:27:02	98
1399	2000	25.8	2025-12-14 05:27:04	99
1400	2000	25.8	2025-12-14 05:27:06	100
1401	2000	25.8	2025-12-14 05:27:08	101
1402	2000	25.8	2025-12-14 05:27:10	102
1403	2000	25.8	2025-12-14 05:27:12	103

❖ Link video demo: https://drive.google.com/file/d/13oNzh-p-wTaRANv4MUNlid_AmbeINTvL/view?usp=sharing

CHƯƠNG 6: TRIỂN KHAI BÀI 2

ĐỀ 2: Phát triển từ câu 1, xây dựng một hệ thống định vị trong nhà sử dụng quảng bá BLE. Giả sử trong không gian nhà có ít nhất 3 thiết bị BLE được đặt tại các vị trí cố định. Mỗi thiết bị thực hiện quảng bá gói tin chứa hai thông tin chính: định danh của thiết bị và dữ liệu môi trường hoặc dữ liệu cảm biến tại vị trí đó (như nhiệt độ/độ ẩm, Góc nghiêng của MPU6050 hoặc nhịp tim và SpO2 trong máu). Người dùng sẽ sử dụng điện thoại thông minh để quét thông tin từ các thiết bị BLE. Dựa trên dữ liệu thu thập được, xác định vị trí của người dùng trong nhà và hiển thị vị trí đó trên ứng dụng. Đồng thời, người dùng cũng có thể theo dõi thông tin (nhiệt độ và độ ẩm, ... tùy cảm biến) của ba vị trí quảng bá trên giao diện ứng dụng một cách trực quan.



Hình 6.1: Mô phỏng vị trí thiết bị dựa vào các thiết bị quảng bá BLE

Yêu cầu thiết bị:

- a. 3 bộ (BRD4180B Radio Board (EFR32xG21 2.4 GHz 20 dBm) + Wireless Starter Kit Mainboard (BRD4001A)).
- b. 3 (sensor + dây kết nối). (Sinh viên tự chuẩn bị).
- c. Điện thoại thông minh hỗ trợ BLE + Ứng dụng sinh viên tự viết để tính toán vị trí của người dùng.

Mô tả chi tiết mô hình định vị trong nhà sử dụng quảng bá BLE:

- Thiết bị quảng bá BLE: Hệ thống gồm 3 thiết bị quảng bá BLE được đặt tại các vị trí cố định trong nhà. Các thiết bị này liên tục thu thập dữ liệu từ cảm biến (nhiệt độ và độ ẩm, ..) sau đó phát quảng bá gói tin theo chu kỳ.
- Người dùng và ứng dụng quét BLE: Người dùng sử dụng điện thoại thông minh để quét thông tin từ 3 thiết bị quảng bá BLE. Dựa trên dữ liệu nhận được, điện thoại sẽ tính toán khoảng cách từ mỗi thiết bị, xác định vị trí của người dùng trong không gian nhà, và hiển thị thông tin này trên ứng dụng. Trên giao diện ứng dụng, vị trí hiện tại của người dùng trong nhà sẽ được hiển thị cùng với thông tin nhiệt độ và độ ẩm tại 3 vị trí đặt các thiết bị BLE.
- Xây dựng bản đồ và chia không gian: Ngôi nhà được chia thành một ma trận các tọa độ, và mỗi phòng được ánh xạ tương ứng với một tập hợp tọa độ xác định. Ví dụ, trong hình minh họa: o Phòng 1: Gồm các tọa độ (x, y) với $x = [0, 1, 2, 3]$ và $y = [0, 1, 2, 3]$. o Phòng 2, 3, 4: Được phân chia tương tự với các tọa độ khác nhau trong ma trận. Khi xác định được tọa độ hiện tại của người dùng, ví dụ (3, 6), hệ thống sẽ đối chiếu với bản đồ ánh xạ để kết luận rằng người dùng đang ở Phòng 3.

1. Yêu cầu bài toán

- **Mục tiêu:** Định vị vị trí người dùng dựa trên 3 thiết bị BLE cố định (Anchor Nodes).
- **Chức năng:**
 - ✓ Các Node quảng bá thông tin định danh và dữ liệu môi trường (Nhiệt độ/Độ ẩm từ 3 cảm biến DHT11).
 - ✓ Thiết bị di động quét tín hiệu, tính toán khoảng cách và xác định vị trí trong không gian được chia lưới (phòng).

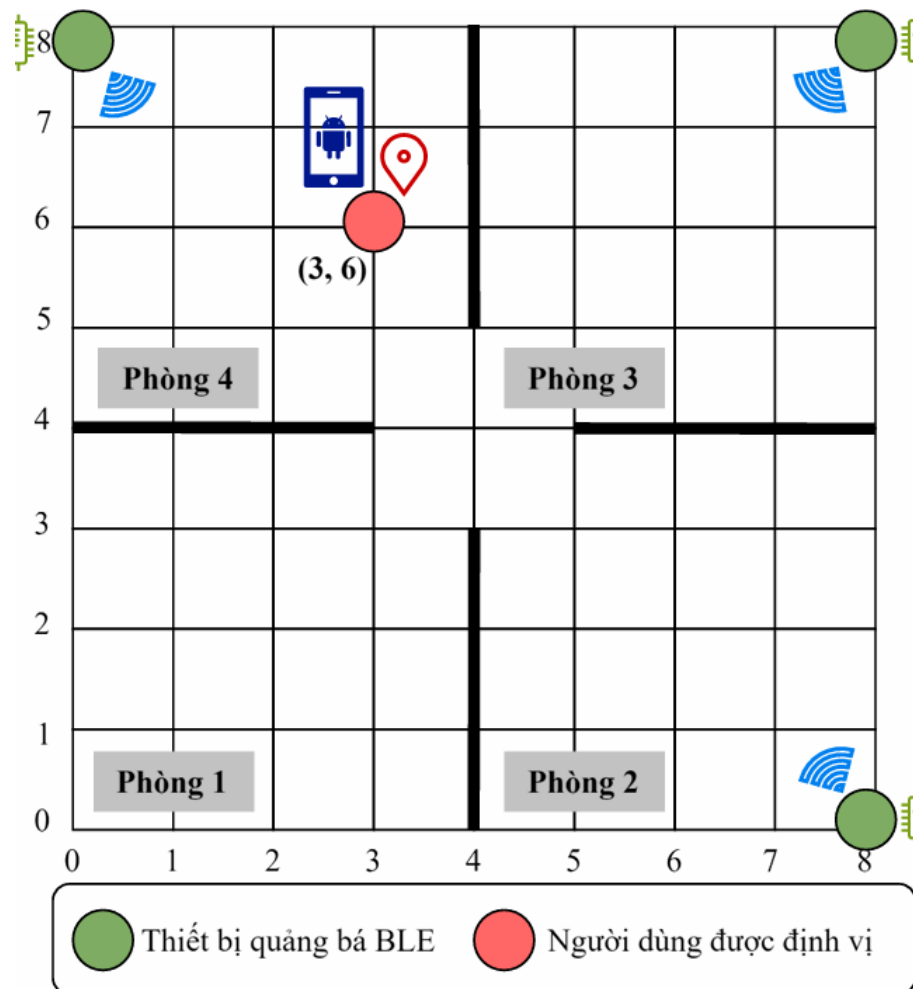
2. Mô hình hệ thống định vị

a. Mô tả không gian:

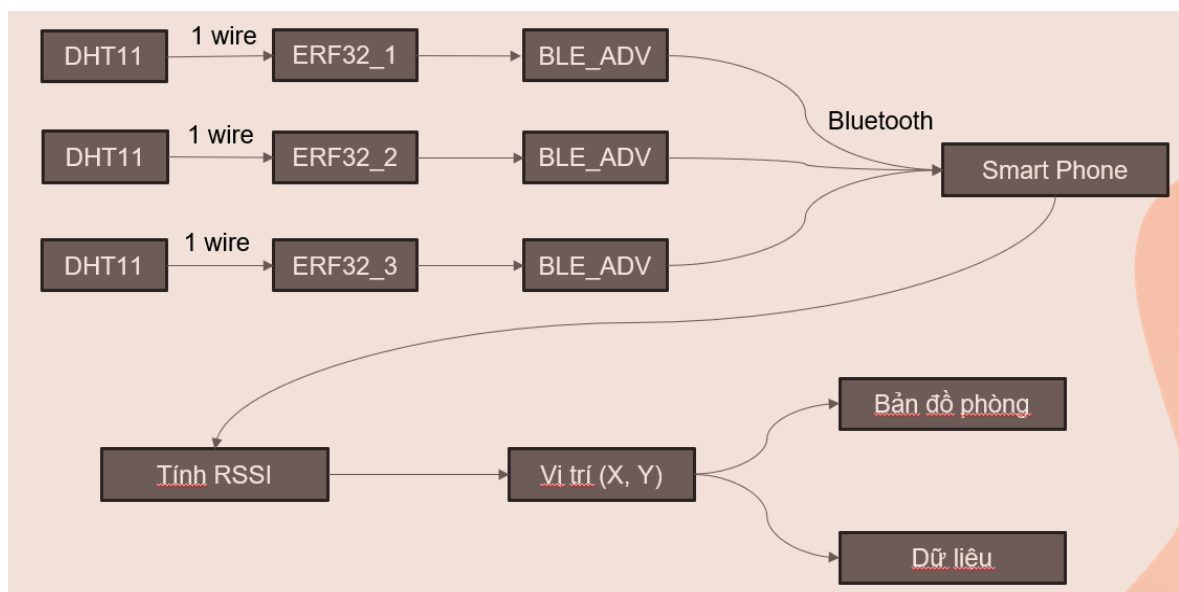
- Sơ đồ bố trí 3 Node tại 3 vị trí cố định (Ví dụ: Node 1 tại (0,0), Node 2 tại (8,0), Node 3 tại (8,8)).
- Quy ước phân chia phòng (Ma trận tọa độ).

b. Nguyên lý định vị:

- Giải thích ngắn gọn về RSSI (Received Signal Strength Indicator).
- Công thức quy đổi RSSI sang khoảng cách (Distance).
- Thuật toán Tam giác lượng (Trilateration) hoặc phương pháp Min-Max/Trọng số để tìm tọa độ (x, y) của điện thoại.



Hình 6.2: Mô phỏng vị trí thiết bị dựa vào các thiết bị quảng bá BLE



Hình 6.3: Block diagram cho câu 2

3. Triển khai hệ thống

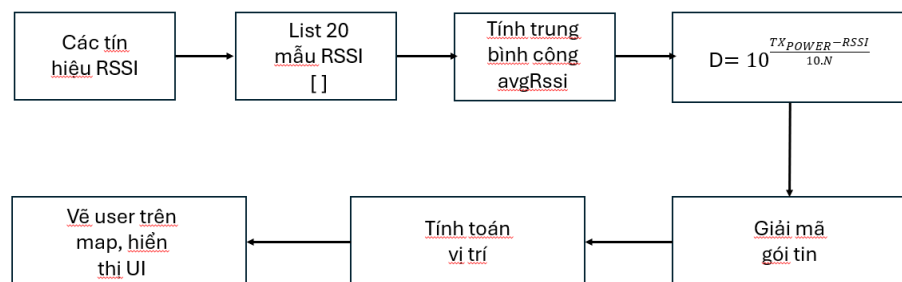
a. Thiết lập các Node quảng bá (Anchor Nodes):

- Sử dụng 3 bộ Kit + 3 cảm biến DHT11.
- Cấu hình mỗi Node có một ID riêng biệt trong gói tin quảng bá để App phân biệt.
- Code trích xuất dữ liệu từ DHT11 đưa vào gói tin BLE.

b. Xây dựng Ứng dụng di động (Mobile App):

- **Công cụ phát triển:** Android Studio - Flutter
- **Giải thuật tính toán vị trí:**

➤ Lưu đồ:



➤ Tạo 1 class BeaconNode:

```
class BeaconNode {  
    String id;          // Tên hiển thị (Trạm 1, 2...)  
    final String macAddr; // Địa chỉ MAC nhận diện  
    final double x;      // Tọa độ mét (X)  
    final double y;      // Tọa độ mét (Y)  
    final Color color;   // Màu sắc hiển thị  
  
    // Dữ liệu thay đổi liên tục  
    int rssi = -100;  
    double distance = 10.0;
```



```
double temp = 0.0;
double hum = 0.0;
DateTime lastUpdate = DateTime.now();

// Danh sách lưu 20 giá trị RSSI gần nhất
List<int> rssiHistory = [];

//Constructor
BeaconNode({
    required this.id,
    required this.macAddr,
    required this.x,
    required this.y,
    required this.color
});
}
```

➤ Khởi tạo 2 thông số trong RSSI:

```
//Đây là cường độ tính hiệu RSSI chuẩn tại 1 mét( dao động từ -51
đến -65)
const int TX_POWER_AT_1M = -59;
//Hệ số môi trường (Khi ở phòng kín từ 2.5 - 3)
const double ENV_FACTOR_N = 3.0;
```

➤ Hàm scan liên tục kết nối các thiết bị BLE:

```
try {
  await FlutterBluePlus.startScan(
    timeout: null, // Quét liên tục
    androidScanMode: AndroidScanMode.LowLatency,
  );
  setState(() => isScanning = true);
} catch (e) {
  print("!!! LỖI SCAN: $e");
}

_scanSubscription = FlutterBluePlus.scanResults.listen((results)
{
  for (ScanResult r in results) {
    try {
      //Lọc ra 3 trạm khớp với địa chỉ mac
      var node = anchors.firstWhere((a) => a.macAddr ==
r.device.remoteId.toString());

      // Xử lý gói tin và tính toán
      _processPacket(node, r);

      // Cập nhật vị trí lên màn hình
      if (mounted) _calculateUserPosition();
    } catch (e) {
      // Bỏ qua thiết bị lạ
    }
  }
}
);
```

- Hàm xử lý gói tin: sử dụng thuật toán lọc trung bình (moving average filter)

```
void _processPacket(BeaconNode node, ScanResult r) {  
  
    // Thêm RSSI mới vào list 20 mẫu  
    node.rssiHistory.add(r.rssi);  
  
    // Chỉ giữ lại 20 mẫu mới nhất để tính toán, nếu vượt quá thì  
    xóa tại vị trí 21  
    if (node.rssiHistory.length > 20) {  
        node.rssiHistory.removeAt(0);  
    }  
  
    // Tính trung bình cộng  
    double sum = node.rssiHistory.fold(0, (p, c) => p + c);  
    double avgRssi = sum / node.rssiHistory.length;  
  
    // Cập nhật giá trị đã lọc vào node (  
    node.rssi = avgRssi.toInt();  
    node.lastUpdate = DateTime.now();  
    //////////////////////////////////////
```

$$D = 10^{\frac{TX_{POWER} - RSSI}{10 \cdot N}}$$

```
////////////////////////////////////
```

```
// Tính số mũ (Exponent)
double exponent = (TX_POWER_AT_1M - avgRssi) / (10 *
ENV_FACTOR_N);
// Tính lũy thừa cơ số 10
double rawDistance = pow(10, exponent).toDouble();

// Nếu tính ra > 12 mét (lớn hơn đường chéo phòng) thì ép về
12m
// Giúp chấm đo không bị bay ra khỏi bản đồ khi nhiều
if (rawDistance > 12.0) rawDistance = 12.0;

node.distance = rawDistance;
//Giải mã gói tin
if (r.advertisementData.manufacturerData.isNotEmpty) {

r.advertisementData.manufacturerData.forEach(
(companyId, bytes) {
// có 4 byte data
if (bytes.length >= 4) {
// [TempInt, TempDec, HumInt, ID]

int tInt = bcdToDecimal(bytes[0]); //Temp phần nguyên
int tDec = bcdToDecimal(bytes[1]); //Temp phần thập
phân

int hInt = bcdToDecimal(bytes[2]); //hump

int deviceId = bytes[3]; // ID

node.id = "Trạm $deviceId";
node.temp = tInt + (tDec / 10.0);
```

```
        node.hum = hInt.toDouble();  
    }  
});  
}  
}
```

- Hàm tính toán vị trí: sử dụng thuật toán Định vị trọng tâm có trọng số (Càng gần thì trọng số càng cao):

✓ Công thức :

$$W = \frac{1}{D^2}$$

```
void _calculateUserPosition() {  
  
    double tuSoX = 0, tuSoY = 0, mauSo = 0;  
    int activeNodes = 0;  
    DateTime now = DateTime.now();  
  
    // Duyệt từng trạm  
    for (var node in anchors) {  
        // Chỉ chấp nhận trạm online trong 15s  
        if (now.difference(node.lastUpdate).inSeconds < 15) {  
  
            activeNodes++;  
  
            // Thuật toán Trọng số(Cộng 0.1 vì tránh chia cho 0)  
  
            double weight = 1.0 / (pow(node.distance, 2) + 0.1);  

```

```
//tổng hợp lực:
//////////#####////////////////

$$X = \frac{x1.w1 + x2.w2 + x3.w3}{w1 + w2 + w3}$$


$$Y = \frac{y1.w1 + y2.w2 + y3.w3}{w1 + w2 + w3}$$

//////////#####////////////////

    tuSoX += node.x * weight;
    tuSoY += node.y * weight;
    mauSo += weight;

}
}

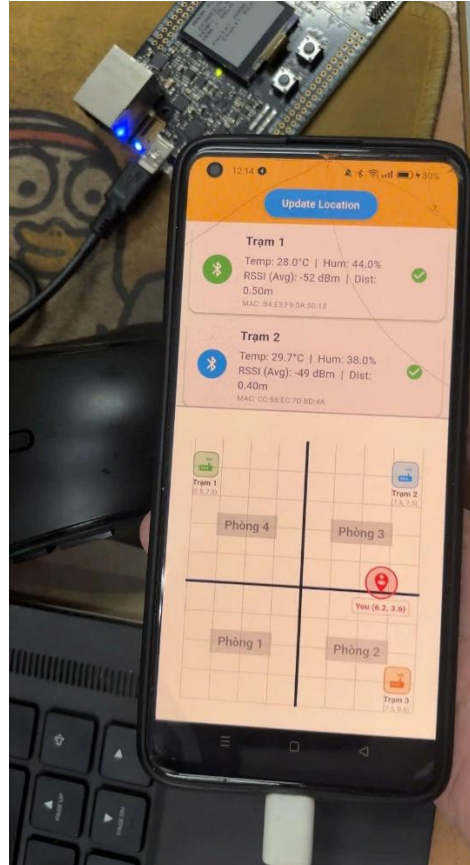
if (activeNodes >= 1 && mauSo > 0) {
    setState(() {
        userX = tuSoX / mauSo;
        userY = tuSoY / mauSo;
    });
}
}
```

- **Ánh xạ vị trí:** Logic để từ tọa độ (x,y) suy ra người dùng đang ở Phòng 1, 2, 3 hay 4.

4. Kết quả thực nghiệm

- **Giao diện App:**

- ✓ Hiển thị thông tin nhiệt độ/độ ẩm của 3 trạm.
- ✓ Hiển thị vị trí người dùng trên bản đồ/lưới tọa độ.



- **Đánh giá độ chính xác:**
 - ✓ Thử nghiệm tại các vị trí đã biết.
 - ✓ So sánh vị trí thực tế và vị trí trên App.
 - ✓ Nhận xét về độ ổn định của RSSI và các yếu tố gây nhiễu.

❖ Link video demo:

<https://drive.google.com/file/d/1doyaW2wTsV4CkWZOw9Z5PYmWK-qD6DKs/view?usp=sharing>

CHƯƠNG 7: KẾT LUẬN - ĐÁNH GIÁ

Sau quá trình nghiên cứu và thực hiện đồ án môn học "Thực hành Giao tiếp máy tính và thu nhận dữ liệu", nhóm đã hoàn thành các yêu cầu đặt ra của Câu 1 và Câu 2, cụ thể như sau:

1. Đối với hệ thống đo nhiệt độ - độ ẩm (Câu 1):

- **Phần cứng:** Kết nối thành công cảm biến DHT11 với kit EFR32xG21. Hệ thống hoạt động ổn định, đọc được thông số nhiệt độ và độ ẩm môi trường.
- **Hiển thị tại chỗ:** Dữ liệu hiển thị rõ ràng trên màn hình LCD của kit, cập nhật theo chu kỳ
- **Giao tiếp PC:** Xây dựng thành công ứng dụng trên PC, nhận dữ liệu qua UART, hiển thị trực quan và lưu trữ được vào cơ sở dữ liệu
- **Giao tiếp BLE:** Thiết bị thực hiện quảng bá (Advertising) gói tin chứa dữ liệu cảm biến chuẩn xác để các thiết bị khác có thể quét được.

2. Đối với hệ thống định vị trong nhà (Câu 2):

- **Hạ tầng mạng:** Triển khai thành công 3 Anchor Node tại các vị trí cố định, mỗi node đều gửi kèm dữ liệu môi trường riêng biệt.
- **Ứng dụng di động:** Xây dựng hoàn thiện ứng dụng trên Smartphone có khả năng:
 - ✓ Quét đồng thời 3 thiết bị BLE.
 - ✓ Trích xuất dữ liệu nhiệt độ/độ ẩm của từng phòng.
 - ✓ Thu thập giá trị RSSI và thực hiện thuật toán định vị.
- **Tính năng định vị:** Xác định được vị trí tương đối của người dùng và hiển thị đúng vị trí trên sơ đồ phòng (Map) đã thiết lập

3. Đánh giá hệ thống

a. Ưu điểm:

- **Độ ổn định:** Hệ thống hoạt động liên tục trong thời gian dài [ví dụ: 30 phút] mà không xảy ra hiện tượng treo hay mất kết nối.
- **Chi phí:** Việc sử dụng cảm biến DHT11 thay cho các cảm biến I2C đắt tiền giúp tối ưu chi phí nhưng vẫn đảm bảo được yêu cầu thu thập dữ liệu của đề tài.
- **Giao diện:** Phần mềm trên PC và App điện thoại có giao diện thân thiện, dễ sử dụng, đáp ứng tốt nhu cầu giám sát thời gian thực.

b. Nhược điểm và hạn chế:

- **Độ chính xác của RSSI:** Tín hiệu RSSI của BLE bị dao động khá mạnh và dễ bị ảnh hưởng bởi vật cản (tường, người đi lại) và nhiễu môi trường, dẫn đến sai số trong việc tính toán khoảng cách. Vị trí hiển thị trên App đôi khi còn bị "nhảy" giữa các vùng lân cận.
- **Tốc độ phản hồi:** Cảm biến DHT11 có độ trễ nhất định (khoảng 1-2 giây để đọc), nên chưa đáp ứng được các ứng dụng cần tốc độ cực nhanh.

4. Bảng đánh giá thành viên trong nhóm:

HỌ VÀ TÊN	MSSV	MỨC ĐỘ HOÀN THÀNH
Nguyễn Vũ Nhật Thành	22200145	TỐT
Nguyễn Đức Thuận	22200158	TỐT
Lê Đăng Tâm	22200140	TỐT
Nguyễn Thế Vỹ	22200186	TỐT

PHIẾU ĐÁNH GIÁ ĐỒ ÁN THỰC HÀNH GIAO TIẾP MÁY TÍNH VÀ THU NHẬN DỮ LIỆU

I. Phần bắt buộc

Tiêu chí	Điểm	Tổng điểm tối đa	Đánh giá
Demo hoạt động tốt	1		
Mức độ hoàn thiện và độ tin cậy của dữ liệu	1		
Giải thích nguyên lý hoạt động	1		
Sử dụng ngắt, timer, uart, adc, ...	0,5		
Sử dụng cơ sở dữ liệu	0,5		
Cuốn báo cáo trình bày đầy đủ nội dung	0.5		
Slide trình bày dễ hiểu	0.5		

II. Phần điểm cộng

Tiêu chí	Điểm	Tổng điểm tối đa	Đánh giá
Demo hoạt động tốt	0,5		
Giải thích nguyên lý và thuật toán	0,5		
Mức độ hoàn thiện và độ tin cậy của dữ liệu	1		

STT Nhóm:

Điểm tổng kết:

....., ngày ... tháng ... năm 2025

Người đại diện nhóm
(Ký tên, ghi rõ họ tên)